Create a web application for a **simple blog system** using the **MERN stack (MongoDB, Express.js, React.js, Node.js)** with user authentication, post management, and comment functionality.
The application should also include an **admin panel** for managing users and posts.

Requirements

1. User Authentication

- Implement **user registration, login, and logout** functionality.
- Implement **JWT access tokens** and **refresh tokens**
- Protect authentication endpoints using **rate limiting**
- Store sensitive credentials using **environment variables (.env)**
- Implement **secure password hashing** (e.g., bcrypt).
- Include **social media login**:
  - o Google
  - o Facebook
  - (using OAuth 2.0 via libraries)

2. User Roles and Permissions

- Implement two types of users:
  - o **Admin**
  - o **Regular User**
- **Admins** can:
  - o Manage all users
  - o Manage all posts
- **Regular users** can:
  - o Create, edit, and delete **only their own posts**
- Implement **role-based access control (RBAC)** using middleware.
  - o Enforce role-based access **at the API level** (not only on the frontend)

3. Post Management

- Implement **CRUD operations** for blog posts.
- Each post should include:
  - o Title
  - o Content
  - o Author
  - o Timestamps
- Validate request payloads using a validation library (e.g., Joi / Zod)
- Generate a URL-friendly slug for each post
- Use **MongoDB with Mongoose ODM** for database interactions.
- Implement **soft deletes** for posts (e.g., isDeleted flag or deletedAt field).

4. Comments

- Users can **comment on posts**.
- Implement **CRUD operations** for comments.
- Users can edit and delete **only their own comments**; Admins can manage all comments
- Use **Mongoose relationships/references** to manage post–comment associations.

5. Admin Panel

- Create a dedicated **Admin Panel** (React-based UI).
- Restrict access using **authentication and role-based middleware**.
- Implement an **admin dashboard** displaying:
  - o Total number of users
  - o Total number of posts
  - o Total number of comments

6. Advanced Routing

- Use **Express Router** to group routes logically.
- Protect routes using **middleware**.
- Implement **RESTful APIs** with proper naming conventions (clean response structure, versioning).
- Implement centralized error handling with consistent API error responses

7. Middleware

- Create custom middleware to:
  - o Log user activities (e.g., login, post creation, deletion).
  - o Validate JWT tokens.
  - o Check user roles and permissions.
- Ensure middleware is reusable and modular.

8. Services & Architecture

- Create a **service layer** to handle business logic (separate from controllers).
- Follow a **clean architecture**
- Ensure proper separation of concerns.
- Follow a clear folder structure (controllers, services, models, routes, middleware)

9. Performance Optimization

- Optimize MongoDB queries using:
  - o Indexing
  - o Pagination
- Use **data population wisely** to avoid unnecessary database calls.

10. Testing

- Write **unit and integration tests**

- Use testing tools such as:
    - Jest
- Ensure good test coverage and maintainable test cases.

Frontend Requirements (React)

- Use functional components and React Hooks
- Handle authentication state globally (Context API or equivalent)
- Implement protected routes for authenticated and admin users

Bonus (Optional)

- Implement **real-time notifications** using:
    - Socket.io (or similar real-time libraries)

Deliverables

- A **GitHub repository** containing the complete MERN-based source code.
- A **README.md** file including:
    - Installation instructions
    - Environment setup
    - Feature explanations
    - API documentation overview
- A **5–10 minute demo video** showcasing:
    - Application flow
    - Admin panel
    - Key features

Evaluation Criteria

- Code quality and adherence to best practices.
- Proper use of **MERN stack technologies**.
- Clean architecture and modular design.
- Clear documentation and maintainability.